PROJECT REPORT

CHAINGUARD

The Digital Black Box for Sensitive Shipments

"Turning shipment telemetry into interpretable condition risk and actionable decisions."

DOMAIN

IoT & Embedded Smart Systems

TEAM

Diazonium

Team Members

Ananya Joshi	Frontend, Backend & Deployment
Vaibhav	Documentation & Pitch
Utkarsh	Decision Engine & Mathematical Model
Unnat	Hardware & ESP32 Integration

Live Dashboard: https://chain-guard-ruddy.vercel.app/
Repository: https://github.com/ananyajoshi-cseai/ChainGuard

Hackathon Prototype Submission

TABLE OF CONTENTS

Contents

1 Introduction

1.1 Problem Statement

Modern logistics networks track the *location* of a shipment with high precision, yet remain almost entirely blind to its *condition*. A parcel may traverse multiple handling points, vehicles, and storage facilities and arrive with an intact exterior while having silently experienced temperature or humidity excursions, mechanical shocks from rough handling, or unauthorized access to its enclosure. For temperature-sensitive pharmaceuticals, biological samples, precision electronics, or high-value fragile goods, this gap between *location certainty* and *condition uncertainty* is the single largest unresolved risk in the last-mile and mid-mile supply chain.

The Tracking Gap

Conventional GPS/RFID tracking answers the question *"Where is the package?"* It does not answer the operationally critical question: *"What happened to the package during transit, and does its current condition warrant inspection before acceptance?"*

1.2 Motivation

The inspiration for ChainGuard is drawn from the aviation *flight data recorder* ("black box") paradigm: a self-contained instrumentation unit that continuously observes the physical environment of a critical asset, timestamps every deviation from nominal operating conditions, and preserves that record independent of the outcome. Applied to logistics, the same principle allows a receiving party to reconstruct the condition history of a shipment rather than relying solely on visual inspection at the point of delivery, which cannot detect latent thermal, mechanical, or tamper-related damage.

1.3 Objectives

- Instrument a shipment with low-cost embedded sensors capable of continuously observing temperature, humidity, acceleration (shock), and enclosure integrity.
- Transmit sensor telemetry over a wireless link to a cloud-hosted backend in near real time.
- Convert raw, noisy telemetry into discrete, semantically meaningful *events* using a stateful detection process rather than naive threshold alarms.
- Aggregate detected events into a single, interpretable **Integrity Score** using a transparent, explainable risk model rather than an opaque classifier.
- Surface the score, its constituent risk contributions, and the full event history on a live monitoring dashboard to support an operational **ACCEPT / INSPECT / HIGH RISK** decision.

2 Conceptual Positioning

2.1 Location Tracking vs. Condition Monitoring

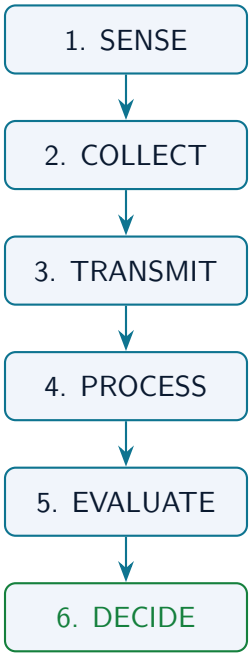
It is useful to separate supply-chain visibility into two orthogonal concerns, summarized in Table 1.

2.2 Design Philosophy: SENSE → COLLECT → TRANSMIT → PROCESS → EVALUATE → DECIDE

ChainGuard is organized as a strict linear pipeline. Each stage consumes the output of the previous stage and exposes a narrow, well-defined interface to the next, which keeps the edge firmware simple (it only senses, packages and transmits) while concentrating all interpretive intelligence in the cloud backend, where it can be iterated on without re-flashing hardware in the field.

Dimension	Location Tracking	Condition Monitoring (ChainGuard)
Primary question	Where is the asset?	What state is the asset in?
Typical sensors	GPS, RFID, cellular triangulation	Temperature, humidity, IMU (shock), tamper switch
Failure visibility	Loss / delay	Thermal excursion, physical shock, tampering
Output	Coordinates, ETA	Risk-scored decision (ACCEPT/INSPECT/HIGH RISK)

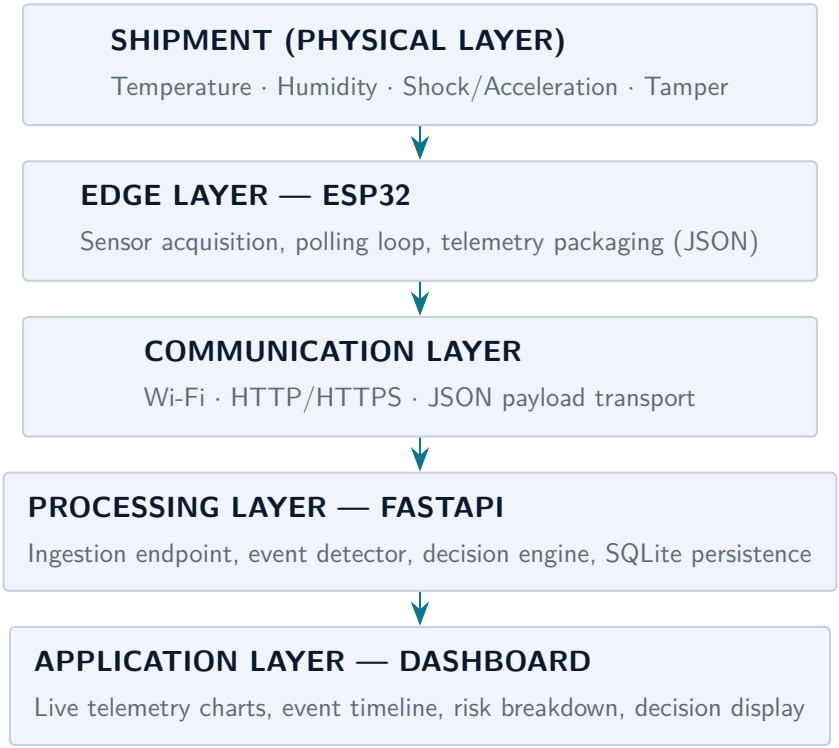
Table 1: Positioning ChainGuard relative to conventional tracking systems.



3 System Architecture

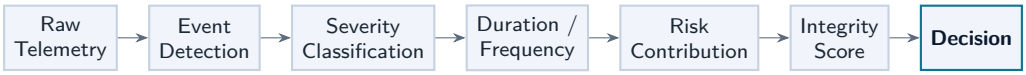
3.1 Layered View

ChainGuard is structured into four architectural layers, each independently deployable and independently testable, which is a deliberate separation-of-concerns choice that lets the hardware team, backend team, and frontend team iterate in parallel.



3.2 Logical Data Transformation Pipeline

While Section 3.1 shows the physical / infrastructural layering, Figure below shows the *logical* transformation the data undergoes as it moves from a raw floating-point sensor sample to a discrete operational decision.



EDGE: ESP32 + Sensors | **COMMUNICATION:** Wi-Fi + HTTP/HTTPS + JSON | **PROCESSING:** FastAPI + SQLite + Event Detector + Decision Engine | **APPLICATION:** Dashboard + Charts + Risk + Events

4 Hardware Subsystem

4.1 Component Selection

The edge device is built around the ESP32 family of microcontrollers, chosen for its integrated dual-core processor, native Wi-Fi radio, and low unit cost, all of which are important constraints for a disposable or semi-disposable in-transit sensing unit.

Component	Role	Justification
ESP32 / ESP32-S3	Microcontroller & Wi-Fi radio	Integrated Wi-Fi removes the need for a separate networking module; sufficient GPIO/I2C for all sensors; low power sleep modes for future battery operation.
DHT11	Temperature & Humidity	Single-wire digital sensor, simple integration, adequate resolution for excursion detection at prototype scale.
MPU6050	3-axis Accelerometer / Shock	I2C interface, provides acceleration magnitude used to derive shock events and their severity.
Reed Switch	Tamper / Enclosure-open Detection	Simple magnetic digital switch: enclosure separation breaks the magnetic circuit, producing an unambiguous binary tamper flag.
Wi-Fi (on-board)	Telemetry Uplink	Enables direct HTTP POST to the cloud backend without an intermediate gateway.

Table 2: Hardware Bill of Materials and design rationale for the sensing edge.

4.2 Circuit Design and Simulation

Prior to any physical prototyping, the sensor wiring, power distribution, and I2C/digital bus topology were designed and functionally validated in **Cirkit Designer**, a browser-based circuit simulation environment. This allowed the team to verify sensor addressing, pin conflicts, and polling logic entirely in software before committing to physical assembly — a standard hardware/software co-design practice that reduces iteration cost.

Cirkit Designer Simulation Workspace
https://app.cirkitdesigner.com/project/80530e4d-29f7-445a-b491-e911d28aa472

4.3 Firmware Behaviour

The firmware (`hardware/chainguard_esp32.ino`) implements a straightforward acquire–package–transmit loop:

Firmware Main Loop (Pseudocode)

```
loop():
    t, h      <- read_DHT11()
    ax,ay,az  <- read_MPU6050()
    a_mag     <- magnitude(ax, ay, az)
    tamper    <- read_reed_switch()
    payload   <- {
```

```
        shipment_id: CONFIG.ID,
        temperature: t,
        humidity: h,
        acceleration: a_mag,
        tamper_status: tamper
    }
    POST /api/telemetry  (payload)  -> backend
    sleep(POLL_INTERVAL_MS)
```

The firmware itself carries no interpretive logic: it does not decide whether a reading is anomalous. This is an intentional architectural boundary — keeping the edge "dumb" and the cloud "smart" means detection thresholds and risk weights can be revised centrally without re-flashing every deployed unit.

5 Software Architecture

5.1 Backend Module Structure

The backend is a Python service built on **FastAPI** (served by Uvicorn) with **Pydantic** request/response validation and a **SQLite** persistence layer. Its responsibilities are cleanly separated across four modules.

Module	Responsibility
`main.py`	FastAPI application instance, route declarations, request orchestration for the telemetry ingestion flow.
`database.py`	SQLite schema definition, telemetry/event storage, and shipment state persistence.
`event_detector.py`	Stateful lifecycle management of condition excursions — opening, tracking, and closing events.
`decision_engine.py`	Severity classification, risk weighting, duration/frequency amplification, and Integrity Score computation.

Table 3: Backend module decomposition.

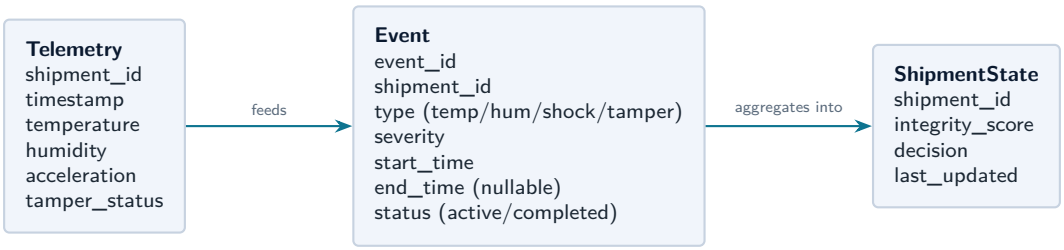
5.2 Repository Layout

Repository Structure

```
ChainGuard/
|-- backend/
|   |-- main.py              # FastAPI app, endpoints, ingestion flow
|   |-- database.py          # SQLite schema, storage, state management
|   |-- decision_engine.py   # Severity, risk, Integrity Score logic
|   |-- event_detector.py    # Active/completed event lifecycle
|   `-- requirements.txt     # Python dependencies
|
|-- frontend/
|   |-- index.html           # Dashboard UI structure
|   |-- script.js            # API integration, charts, state management
|   `-- style.css            # Dashboard styling
|
|-- hardware/
|   `-- chainguard_esp32.ino # ESP32 firmware for sensor polling & HTTP POST
|
|-- .gitignore
`-- README.md
```

5.3 Data Model

Conceptually, the SQLite layer persists three related entities: raw *telemetry samples*, derived *events* (with active/completed lifecycle state), and a per-shipment *current state* snapshot used to serve the dashboard without recomputing history on every request.



5.4 API Design

The backend exposes a small, purpose-built REST surface. Every route is documented and interactively testable through the auto-generated Swagger UI.

Method	Endpoint	Description
GET	`/`	Root status message.
GET	`/health`	API health check (used by uptime monitoring).
POST	`/api/telemetry`	Ingests a JSON telemetry payload from the ESP32, triggers event detection, recalculates risk, and returns the updated shipment state.
GET	`/api/telemetry/latest/{shipment_id}`	Retrieves the most recent telemetry reading for a shipment.
GET	`/api/telemetry/{shipment_id}`	Retrieves the full historical telemetry series for a shipment.
GET	`/api/events/{shipment_id}`	Lists all completed (closed) events.
GET	`/api/events/{shipment_id}/active`	Lists currently ongoing condition excursions.
GET	`/api/shipments/{shipment_id}/summary`	Comprehensive view: Integrity Score, decision, active events, and recent telemetry, in one call.

Table 4: ChainGuard REST API surface.

Example: POST /api/telemetry

```
{
  "shipment_id": "CG-1042",
  "temperature": 5.4,
  "humidity": 58.0,
  "acceleration": 1.72,
  "tamper_status": 0
}
```

5.5 Frontend Dashboard

The dashboard is a lightweight, dependency-minimal client (HTML/CSS/vanilla JavaScript with **Chart.js** for time-series rendering), deployed on Vercel. It polls the `/summary` endpoint on an interval and renders: the live Integrity Score, the current ACCEPT/INSPECT/HIGH RISK decision, real-time sensor charts, the active/completed event timeline, and a per-metric risk contribution breakdown so an operator can see *why* the score is what it is rather than trusting a single opaque number.

6 Theoretical Framework: Event Detection

Rather than raising an alarm on every out-of-range sample — which would be brittle to sensor noise and produce an unusable flood of events — ChainGuard models condition monitoring as a **stateful lifecycle process**. This section formalizes that process.

6.1 Formal Definitions

Definition 6.1 — Telemetry Sample

A telemetry sample is a tuple

$$\tau = \langle \text{id}, t, T, H, A, X \rangle$$

where t is the ingestion timestamp, T is temperature (°C), H is relative humidity (%), A is acceleration magnitude (m/s^2), and $X \in \{0, 1\}$ is the binary tamper flag.

Definition 6.2 — Severity Function

For metric $i \in \{T, H, A\}$, a severity function $\sigma_i : \mathbb{R} \to \{0, 1, 2, 3\}$ maps a raw reading to a discrete severity band, where 0 denotes the normal operating range and 3 denotes the most extreme configured deviation. For the tamper flag, $\sigma_X(X) = 3X$ (binary: either normal or maximal severity, with no intermediate band).

Definition 6.3 — Condition Event

A condition event is a tuple

$$e = \langle \text{type}, s_{\text{start}}, s_{\text{end}}, \sigma_{\max}, \text{status} \rangle$$

where type $\in \{\text{TEMP}, \text{HUM}, \text{SHOCK}, \text{TAMPER}\}$, s_{start} and s_{end} are the opening and closing timestamps ($s_{\text{end}} = \perp$ while the event is open), $\sigma_{\max}$ is the highest severity band observed during the event's lifetime, and status $\in \{\text{ACTIVE}, \text{COMPLETED}\}$.

6.2 Threshold Bands

The severity function σ_i is realized in the current prototype configuration as the following threshold bands.

Parameter	Normal (0)	Sev. 1	Sev. 2	Sev. 3
Temperature	2–8 °C	0–2 / 8–10 °C	−2–0 / 10–15 °C	Beyond above
Humidity	30–70%	20–30 / 70–80%	10–20 / 80–90%	Beyond above
Shock	≤ 2.5 m/s^2	≤ 4.0 m/s^2	≤ 6.0 m/s^2	> 6.0 m/s^2
Tamper	0	—	—	1

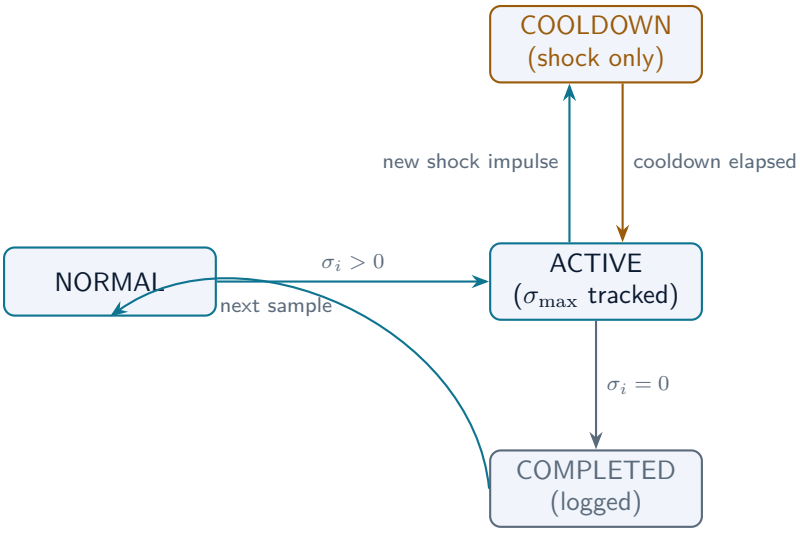
Table 5: Severity band configuration (MVP defaults; tunable per shipment type in future scope).

The thresholds above are configurable prototype values calibrated for a cold-chain-style demonstration profile (e.g. 2–8 °C being a typical vaccine storage band). They are not asserted as validated pharmaceutical, medical, or regulatory transport standards.

6.3 Event Lifecycle Automaton

Each metric's event lifecycle is modeled as a finite state machine. A new event opens the first time a sample's severity exceeds zero; the event remains active while subsequent samples continue to register non-zero severity (with $\sigma_{\max}$ updated to the running maximum); the event closes the first time the metric returns to its normal

band. Shock events additionally pass through a short *cooldown* state to prevent a single physical jolt from being fragmented into many spurious micro-events.



Event Detection Algorithm (Pseudocode)

```
on telemetry_sample(tau):
    for metric i in {TEMP, HUM, SHOCK, TAMPER}:
        sigma_i <- severity(i, tau)
        e <- active_event(shipment_id, i)      # or None

        if sigma_i > 0 and e is None:
            open_event(i, start=tau.t, sigma_max=sigma_i)

        elif sigma_i > 0 and e is not None:
            e.sigma_max <- max(e.sigma_max, sigma_i)
            if i == SHOCK: reset_cooldown(e)

        elif sigma_i == 0 and e is not None:
            close_event(e, end=tau.t, status=COMPLETED)

    recompute_risk(shipment_id)   # -> decision_engine
```

7 The Decision Engine: Risk Quantification and Integrity Scoring

This is the analytical core of ChainGuard. Where Section 6 determines *whether* something anomalous is happening, this section determines *how much it matters*, expressed on a single, bounded, monotonic scale.

7.1 Design Rationale: Why an Explainable Additive Model?

A black-box classifier (e.g. a trained anomaly-detection model) could in principle map telemetry history to a risk label. ChainGuard deliberately avoids this in favour of a transparent, additive, weighted-risk model for three reasons: (i) it requires no training data, which is unavailable for a novel hackathon prototype; (ii) every risk point can be attributed to a specific metric, which is what the dashboard's "risk breakdown" view exposes to the operator; and (iii) thresholds and weights remain human-auditable and tunable per shipment class, which matters for a decision-support tool that a human must ultimately trust.

7.2 Base Severity Weights

Each metric type is assigned a fixed base weight w_i, reflecting the team's judgement of its relative operational importance — a breached enclosure ($w = 15$) is treated as more consequential than a single humidity excursion ($w = 3$).

Metric	Base Weight w_i
Tamper	**15**
Temperature	8
Shock	7
Humidity	3

Table 6: Base severity weights used in the risk model.

7.3 Amplification Mechanisms

A raw base weight alone would treat a 30-second temperature blip identically to a 3-hour thermal excursion, which is operationally wrong. The decision engine therefore amplifies the base contribution of each active/completed event along two independent axes.

Definition 7.1 — Duration Amplification

For duration-sensitive metrics (temperature, humidity), let δ_i be the elapsed time an event has remained active. The duration multiplier is

$$D_i(\delta_i) \;=\; 1 + \alpha \cdot \min\!\left(\frac{\delta_i}{\delta_{\max}},\, 1\right), \qquad \alpha > 0,$$

so that a momentary excursion contributes close to its base weight, while a sustained excursion approaches $(1+\alpha)\times$ its base weight, saturating at the cap $\delta_{\max}$ to prevent unbounded growth from a single stuck sensor reading.

Definition 7.2 — Frequency Amplification

For shock events, let n_i be the number of discrete shock impulses recorded within a rolling observation window W (gated by the cooldown mechanism of Section 6.4, which prevents one impulse from being double-counted). The frequency multiplier is

$$F(n_i) \;=\; 1 + \beta \cdot \min\!\left(\frac{n_i}{n_{\max}},\, 1\right), \qquad \beta > 0,$$

so that repeated rough handling is penalized more heavily than an isolated bump.

7.4 Per-Metric Risk Contribution

Combining the base weight, normalized severity, and the applicable amplification term gives the risk contribution of metric i:

Risk Contribution

$$r_{\text{temp}} = w_T \cdot \frac{\sigma_{\max,T}}{3} \cdot D_T(\delta_T) \qquad r_{\text{hum}} = w_H \cdot \frac{\sigma_{\max,H}}{3} \cdot D_H(\delta_H)$$

$$r_{\text{shock}} = w_A \cdot \frac{\sigma_{\max,A}}{3} \cdot F(n_A) \qquad r_{\text{tamper}} = w_X \cdot \mathbb{1}[X = 1]$$

7.5 Cumulative Risk and Integrity Score

The shipment's cumulative risk at time t is the sum of contributions from every event recorded up to and including t:

Integrity Score

$$R(t) \;=\; \sum_{e \,\in\, \text{Events}(t)} r_{\text{type}(e)} \qquad\qquad I(t) \;=\; \text{clamp}\big(100 - R(t),\; 0,\; 100\big)$$

7.6 Decision Function

The Integrity Score is mapped to one of three discrete operational decisions:

Definition 7.3 — Decision Function

$$D(I) = \begin{cases} \textbf{ACCEPT} & I \geq 80 \\ \textbf{INSPECT} & 50 \leq I < 80 \\ \textbf{HIGH RISK} & I < 50 \end{cases}$$

$I \geq 80$	**ACCEPT** — conditions remained within normal/near-normal bounds for the whole transit.
$50 \leq I < 80$	**INSPECT** — one or more anomalies detected; manual review recommended before acceptance.
$I < 50$	**HIGH RISK** — critical condition failure(s) detected; escalation warranted.

7.7 Worked Example

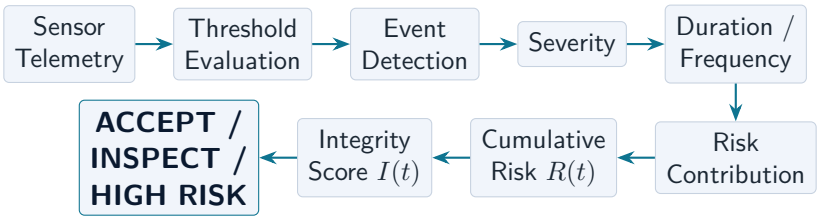
Illustrative Walkthrough

Suppose a shipment experiences: (a) a tamper event ($r_{\text{tamper}} = 15$), and (b) a Severity-2 temperature excursion lasting long enough to reach the duration cap ($\sigma_{\max,T} = 2$, $D_T \approx 1 + \alpha$; taking $\alpha = 0.5$, $r_{\text{temp}} = 8 \cdot \frac{2}{3} \cdot 1.5 = 8.0$). No shock or humidity events occur.

$$R = 15 + 8.0 = 23.0 \qquad I = 100 - 23.0 = 77.0$$

Since $50 \leq 77.0 < 80$, the decision engine reports **INSPECT**, correctly flagging the shipment for manual review despite neither individual event alone crossing the HIGH RISK boundary.

7.8 Decision Engine Data Flow



Disclaimer: ChainGuard is a hackathon prototype for shipment condition monitoring and decision support. Its thresholds, severity levels, weights, and score mappings are configurable prototype logic and are not presented as validated medical, pharmaceutical, regulatory, or safety standards. A HIGH RISK result indicates that the configured monitoring rules detected sufficient condition risk to warrant attention; it does not independently prove product damage.

8 Technology Stack

Hardware

- ESP32 / ESP32-S3
- DHT11 (Temp + Humidity)
- MPU6050 (Accel/Shock)
- Reed switch (Tamper)
- Wi-Fi module

Backend

- Python 3.9+
- FastAPI / Uvicorn
- Pydantic
- SQLite

Frontend & Deploy

- HTML / CSS / JS
- Chart.js
- Vercel (frontend)
- Render (backend API)

Why FastAPI: Automatic request/response validation via Pydantic reduces malformed-telemetry bugs at the ingestion boundary; native async support suits the polling-driven, I/O-bound nature of the workload; and the auto-generated OpenAPI/Swagger UI (`/docs`) doubles as living API documentation for both the hardware and frontend teams during parallel development.

9 System Integration and Deployment

9.1 End-to-End Demonstration Flow

NORMAL PATH: Normal telemetry → ESP32/simulation → FastAPI → event detection (no event opens) → risk unchanged → Integrity Score stable → dashboard shows ACCEPT.

EXCEPTION PATH: Condition event (shock / temperature / tamper) → telemetry values cross a threshold → backend opens/updates an event → risk recalculated → Integrity Score drops → dashboard reflects the updated decision and risk breakdown in real time.

9.2 Deployment Topology

9.3 Local Development Setup

Setup Commands

```
git clone https://github.com/ananyajoshi-cseai/ChainGuard.git
cd ChainGuard

python -m venv .venv
source .venv/bin/activate        # or .venv\Scripts\Activate.ps1 on Windows

cd backend
pip install -r requirements.txt
uvicorn main:app --reload        # http://127.0.0.1:8000  (docs at /docs)

# In a second terminal: serve frontend/index.html via a local dev server
# and point API_BASE in script.js to localhost:8000
```

10 Quick Access

OPEN LIVE DASHBOARD
https://chain-guard-ruddy.vercel.app/

BACKEND API
https://chainguard-qpy6.onrender.com/

SWAGGER DOCS
https://chainguard-qpy6.onrender.com/docs

API HEALTH
https://chainguard-qpy6.onrender.com/health

GITHUB REPO
https://github.com/ananyajoshi-cseai/ChainGuard

OPEN CIRKIT DESIGNER (IoT SIMULATION)
https://app.cirkitdesigner.com/project/80530e4d-29f7-445a-b491-e911d28aa472

11 Results and Current MVP Status

Implemented and Functional

- ✓ ESP32 → deployed backend telemetry flow (HTTP)
- ✓ SQLite-based telemetry and event storage
- ✓ Temp / humidity / shock / tamper detection
- ✓ Active → completed event lifecycle
- ✓ Severity-based risk contribution
- ✓ Duration-aware risk amplification
- ✓ Frequency-aware shock risk amplification
- ✓ Cumulative Integrity Score computation
- ✓ ACCEPT / INSPECT / HIGH RISK decisioning
- ✓ Live, auto-refreshing dashboard with charts
- ✓ Event timeline and risk breakdown view
- ✓ Cloud deployment (Vercel + Render)

12 Limitations and Future Scope

12.1 Known Limitations of the Current Prototype

- Connectivity depends on Wi-Fi availability; there is no offline buffering or cellular fallback for genuine long-haul transit without network coverage.
- SQLite is adequate for a single-shipment demo but is not designed for concurrent multi-shipment, multi-writer production load.
- Thresholds, weights, and amplification constants (α, β, $\delta_{\max}$, $n_{\max}$) are fixed prototype defaults rather than being calibrated per cargo type.

- The tamper mechanism (reed switch) detects enclosure separation but not more sophisticated tampering vectors.

12.2 Future Scope

- GPS / route tracking integration
- Cellular (GSM/LTE-M) connectivity for network-dark transit
- Local microSD buffering for offline logging
- Migration to a persistent cloud database (e.g. PostgreSQL)
- Advanced/ML-based anomaly detection
- Immutable, cryptographically tamper-evident audit trails
- Multi-shipment fleet monitoring dashboard
- Per-shipment configurable thresholds
- Battery and power-budget optimization for long-haul journeys

13 Team Diazonium

Ananya Joshi
Frontend + Backend + Deployment

Utkarsh
Decision Engine + Mathematical Model

Vaibhav
Documentation + Pitch

Unnat
Hardware + ESP32 Integration

14 Conclusion

ChainGuard demonstrates that a small, low-cost embedded sensor package, combined with a deliberately transparent and mathematically explicit risk-scoring backend, can convert raw environmental telemetry into a single, trustworthy, and *explainable* operational decision. The system's core contribution is not any individual sensor reading but the pipeline that turns noisy, continuous physical signals into discrete, auditable events, and those events into a bounded Integrity Score whose every point can be traced back to a specific cause. This closes a meaningful part of the "tracking gap" between knowing where a shipment is and knowing what happened to it — while remaining, by design, simple enough to explain, audit, and extend.

TRACK THE JOURNEY.
UNDERSTAND THE CONDITION.
TRUST THE DECISION.

FINAL LINKS

Live Dashboard:	https://chain-guard-ruddy.vercel.app/
GitHub:	https://github.com/ananyajoshi-cseai/ChainGuard
Swagger Docs:	https://chainguard-qpy6.onrender.com/docs
Cirkit Designer:	https://app.cirkitdesigner.com/project/80530e4d-29f7-445a-b491-e911d28aa472

15 Appendix: Notation Reference

Symbol	Meaning
τ	A single telemetry sample
$\sigma_i(\cdot)$	Severity function for metric i, mapping a reading to $\{0, 1, 2, 3\}$
w_i	Base risk weight for metric i
δ_i	Elapsed active duration of a duration-tracked event on metric i
n_i	Count of impulses for a frequency-tracked event (shock)
$D_i(\cdot)$	Duration amplification multiplier
$F(\cdot)$	Frequency amplification multiplier
r_i	Risk contribution of metric i
$R(t)$	Cumulative risk at time t
$I(t)$	Integrity Score at time t, $\in [0, 100]$
$D(I)$	Decision function mapping score to {ACCEPT, INSPECT, HIGH RISK}

Table 7: Symbol glossary used throughout Sections 6–7.